

Документация проекта Cognitive Engine

Общая информация

Проект Cognitive Engine представляет собой систему для взаимодействия между Unity-приложением, backend API, базой данных PostgreSQL и web-интерфейсом.

Основная цель проекта — приём, хранение и отображение результатов выполнения упражнений, полученных из Unity.

Архитектура проекта:

Unity → ASP.NET Core API → PostgreSQL → React Web Interface

Реализованный функционал

1. Backend API (ASP.NET Core)

Реализован backend на ASP.NET Core.

Основные возможности:

- подключение к PostgreSQL;
- REST API для работы с данными;
- импорт session_result.json из Unity;
- хранение данных пациентов;
- хранение сессий;
- хранение метрик и событий;
- авторизация пользователей;
- работа с ролями пользователей.

Используемые технологии:

- ASP.NET Core;
- Entity Framework Core;
- PostgreSQL;
- Docker.

2. База данных PostgreSQL

Настроена и подключена база данных PostgreSQL.

В БД реализовано хранение:

- пользователей;
- пациентов;
- упражнений;
- сессий;
- событий;
- метрик;
- cursor samples;
- pacer samples;
- body points.

База данных запускается через Docker Compose.

3. Unity → API

Реализован импорт данных из Unity.

Система принимает session_result.json и сохраняет:

- информацию о сессии;
- события Unity;
- итоговые метрики;
- cursor samples;
- pacer samples.

После импорта данные отображаются на web-сайте.

4. Web-интерфейс (React)

Реализован web-интерфейс для работы с системой.

Основной функционал:

- авторизация;
- регистрация пользователей;
- dashboard;
- отображение пациентов;
- отображение сессий;
- отображение метрик;
- просмотр данных импортированных из Unity;
- импорт JSON-файлов;
- создание демо-сессий.

Используемые технологии:

- React;
- JavaScript;

- CSS;
 - Fetch API.
-

Авторизация

Реализована система авторизации пользователей.

Поддерживаются роли:

- оператор;
- методист;
- исследователь;
- администратор.

После входа пользователь получает доступ к соответствующим страницам системы.

Реализованные страницы

Главная страница

Содержит:

- общую информацию;
- переход к API;
- переход к авторизации.

Dashboard

Отображает:

- количество пациентов;
- количество сессий;
- количество упражнений;
- метрики;
- данные Unity.

Список пациентов

Отображает:

- ФИО пациента;
- возраст;
- пол;
- диагноз;
- ограничения;
- статус пациента.

Детализация сессии

Отображает:

- длительность сессии;
 - итоговые метрики;
 - события Unity;
 - samples;
 - данные курсора и расер.
-

Запуск проекта

Backend

Переход в папку backend:

```
cd backend/CognitiveEngine.API
```

Запуск базы данных:

```
docker compose up -d
```

Запуск API:

```
dotnet run
```

API запускается по адресу:

```
http://localhost:5278
```

Frontend

Переход в папку frontend:

```
cd frontend/rc
```

Установка зависимостей:

```
npm install
```

Запуск React-приложения:

```
npm start
```

Frontend запускается по адресу:

```
http://localhost:3000
```

Используемые технологии

Backend

- ASP.NET Core
- Entity Framework Core
- PostgreSQL
- Docker

Frontend

- React
- JavaScript
- CSS

Дополнительно

- Unity
- JSON
- REST API

Архитектура системы

Общая архитектура

Система построена по клиент-серверной архитектуре и состоит из четырёх основных компонентов:

```
Unity Client → ASP.NET Core API → PostgreSQL → React Web Interface
```

Unity

Unity используется как клиентская часть для выполнения упражнений и формирования результатов сессии.

Во время выполнения упражнения Unity:

- отслеживает движение курсора;
- контролирует движение относительно траектории;
- формирует события;
- рассчитывает промежуточные показатели;
- сохраняет результат в `session_result.json`.

После завершения упражнения данные передаются в backend API.

Backend API

Backend реализован на ASP.NET Core.

API является центральным компонентом системы и выполняет следующие задачи:

- приём данных из Unity;
- обработка JSON;
- сохранение данных в БД;
- выдача данных frontend-приложению;
- авторизация пользователей;
- управление пациентами и сессиями.

API работает по REST-архитектуре.

Основные endpoint'ы:

```
/api/auth  
/api/patients  
/api/sessions  
/api/body-points  
/api/exercises  
/api/dashboard
```

PostgreSQL

PostgreSQL используется как основное хранилище данных.

База данных хранит:

- пользователей;
- пациентов;

- упражнения;
- шаблоны упражнений;
- сессии;
- события;
- метрики;
- samples движения;
- body points.

Структура БД разделена на связанные сущности.

Связи между сущностями:

```
Пациент → Сессии → Метрики
                      → Events
                      → CursorSamples
                      → PacerSamples
```

Web-интерфейс

Frontend реализован на React.

Web-интерфейс взаимодействует с backend API через HTTP-запросы.

Основные задачи frontend:

- отображение пациентов;
- отображение результатов;
- отображение метрик;
- авторизация;
- импорт JSON-файлов;
- отображение dashboard.

Логика работы системы

Сценарий работы

1. Авторизация пользователя

Пользователь входит в систему через web-интерфейс.

После авторизации backend возвращает данные пользователя и токен.

В зависимости от роли открываются соответствующие страницы.

2. Выполнение упражнения в Unity

Пациент выполняет упражнение в Unity.

Во время выполнения фиксируются:

- положение курсора;
- отклонения от траектории;
- скорость движения;
- события упражнения;
- временные метрики.

После завершения Unity формирует JSON-файл результата.

3. Импорт результата

JSON-файл отправляется в backend API.

Backend:

- принимает файл;
- валидирует структуру;
- создаёт запись сессии;
- сохраняет события;
- сохраняет метрики;
- сохраняет samples.

После этого данные становятся доступны через frontend.

4. Просмотр результатов

Методист или оператор может:

- открыть пациента;
 - просмотреть список сессий;
 - посмотреть метрики;
 - проанализировать движение;
 - посмотреть историю результатов.
-

Структура проекта

Backend

```
backend/  
└─ CognitiveEngine.API/  
    └─ Controllers/
```



```
|— Dtos/
|— Models/
|— Services/
|— Data/
└─ Program.cs
```

Frontend

```
frontend/
└─ rc/
    │   └─ src/
    │       │   └─ pages/
    │       │   └─ components/
    │       │   └─ styles/
    │       └─ api.js
    └─ public/
```

Database

```
database/
└─ init.sql
└─ docker-compose.yml
```

Текущее состояние проекта

На текущем этапе реализована основная инфраструктура системы:

- backend API;
- база данных;
- импорт данных из Unity;
- web-интерфейс;
- авторизация;
- отображение пациентов и сессий.

Система находится в рабочем состоянии в рамках реализованного функционала.